



VITyarthi

DEVPULSE: SERVER & API HEALTH MONITORING CLI

Academic Project Report & Software Engineering Specification

Course: Programming in JAVA

Project Category: CLI Software Engineering & Distributed Systems Monitoring

Student Name: Mohit Nagar (25BAI11366)

Technology Stack: Pure Java (JDK 11+), Concurrency (CompletableFuture), Git

Architecture: 3-Tier Layered Architecture (Presentation, Service, Data Access)

2. Introduction

Modern backend systems rarely run as a single program anymore. Most production software today is broken up into microservices, REST APIs, and cloud-native components that all talk to each other over a network. That interconnectedness is powerful, but it also means that a single slow dependency, a dropped connection, or a misconfigured endpoint can quietly cascade into a much bigger outage before anyone notices.

Enterprise monitoring platforms such as Datadog, New Relic, or Dynatrace solve this problem well, but they come with real costs: they are heavy, mostly closed-source, need agents installed on every host, and are built around a browser dashboard. That works fine for a full DevOps team, but it is overkill — or simply unavailable — for a developer sitting in an SSH session, a container with no GUI, or a CI runner that needs a quick health check and nothing more.

DevPulse was built to fill that specific gap. It is a small, self-contained Command-Line Interface written entirely in **pure Java (JDK 11+)** with no external dependencies. Under the hood it relies on asynchronous, non-blocking concurrency (CompletableFuture together with `java.net.http.HttpClient`) to ping a whole batch of remote endpoints in parallel, measure their response latency down to the millisecond, work out live SLA uptime numbers, and keep a persistent log of every failure it sees.

3. Problem Statement

Anyone responsible for keeping a set of distributed services healthy runs into the same handful of frustrations sooner or later:

- **Manual checking gets old fast:** Pinging endpoints by hand with `curl` or `ping` works for one server, but it is slow, sequential, and gives you no structured metrics to look back on.
- **No real concurrency in simple scripts:** A basic sequential loop over HTTP requests scales as $O(N)$ with network latency — checking 50 endpoints at 200ms each takes roughly 10 seconds, by which point the numbers are already stale.
- **Nothing is remembered:** Ad-hoc scripts rarely persist anything. There is no saved configuration, no historical min/avg/max latency, and no audit trail of when something actually went down.
- **GUI dashboards are heavy:** Full monitoring dashboards can eat hundreds of megabytes of RAM — not exactly ideal on a small cloud instance or a bare-bones dev container.

DevPulse tries to answer all four points at once: it checks every registered target concurrently, renders the results as a clean, colour-coded table straight in the terminal, tracks SLA percentages across a session, writes failures to a persistent log, and lets you manage the whole list of targets from an interactive menu.

4. Functional Requirements

The application is organised around three tightly-scoped functional modules.

4.1 Module 1 — Service Registry & Configuration (CRUD)

- **Add Endpoint:** Register a new HTTP/HTTPS target with a name, the target URL, an expected status code (200 by default), and a timeout between 1 and 60 seconds.

- **List Endpoints:** Print every registered endpoint as a formatted ASCII table, along with its ID (EP-xxx), URL, and configuration.
- **Update Endpoint:** Edit an existing endpoint's fields, falling back sensibly to the current value if a field is left blank.
- **Delete Endpoint:** Remove a target, with an explicit confirmation step so nothing gets deleted by accident.
- **Persistence:** Every change is written straight to `data/endpoints.csv` using RFC-4180 compliant escaping and an atomic temp-file swap.

4.2 Module 2 — Polling & Metrics Engine

- **Asynchronous Concurrent Polling:** All registered endpoints are probed at the same time through a bounded thread pool and Java's `HttpClient`. The total time for a batch is bounded by the single slowest request, not the sum of all of them.
- **High-Precision Latency Benchmarking:** Round-trip response time is measured with sub-millisecond precision using `System.nanoTime()`.
- **Fault Handling & Error Classification:** Network outcomes are mapped to distinct statuses — UP, DOWN, TIMEOUT, ERROR — and DNS failures, connection refusals, and SSL handshake problems are all caught gracefully instead of crashing the app.

4.3 Module 3 — Analytics, Reporting & Alert Auditing

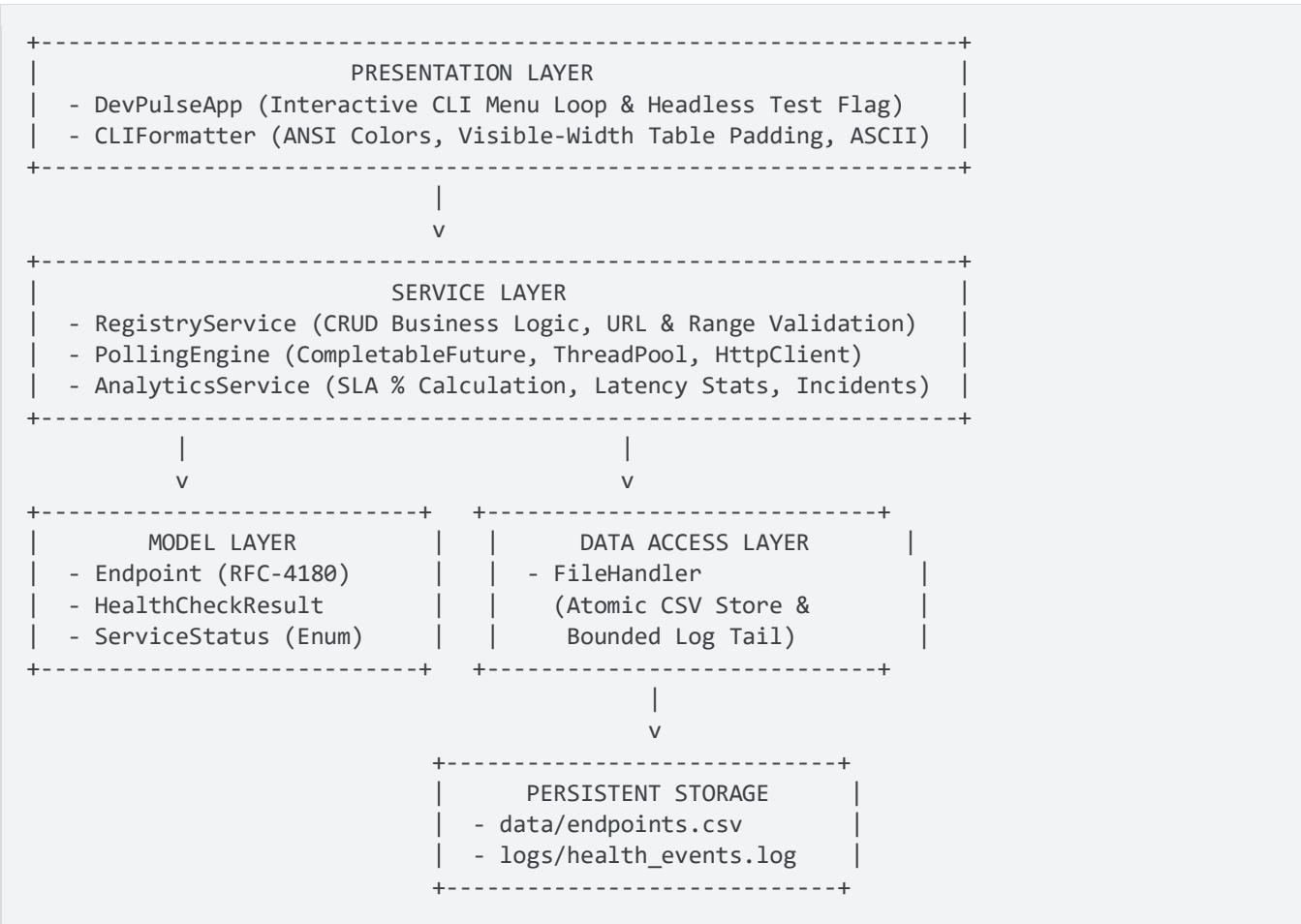
- **Real-Time Terminal Dashboard:** Renders status badges (`[OK]` UP, `[FAIL]` DOWN, `[WARN]` TIMEOUT, `[ERR]` ERROR), HTTP status codes, latency, and diagnostics.
- **SLA Uptime Computation:** Works out the session uptime percentage as $(\text{Successful Checks} \div \text{Total Checks}) \times 100$.
- **Latency Distribution Statistics:** Tracks the minimum, maximum, and average response latency per target across all successful requests.
- **Incident Event Auditing:** Every failure is written, with a timestamp, to `logs/health_events.log`.

5. Non-Functional Requirements

Requirement	Specification & Quality Metric	Implementation Strategy
Performance & Concurrency	Total cycle time should follow $O(\max ti)$ rather than $O(\sum ti)$.	Asynchronous non-blocking worker pool built on <code>CompletableFuture</code> .
Reliability & Fault Tolerance	Zero unhandled crashes during network outages or bad input.	Multi-tiered exception handling (DNS, timeout, SSL, EOF protection).
Data Integrity	Zero data loss or file corruption if the process is interrupted mid-write.	Atomic temp-file writes (<code>.tmp</code> → <code>ATOMIC_MOVE</code>).
Resource Efficiency	$O(1)$ memory overhead and zero external library dependencies.	Standard Java library only; bounded tail-log reading with <code>ArrayDeque</code> .
Portability & Usability	Runs the same way on Windows, Linux, and macOS.	Cross-platform ANSI terminal formatting and pure Java bytecode.

6. System Architecture

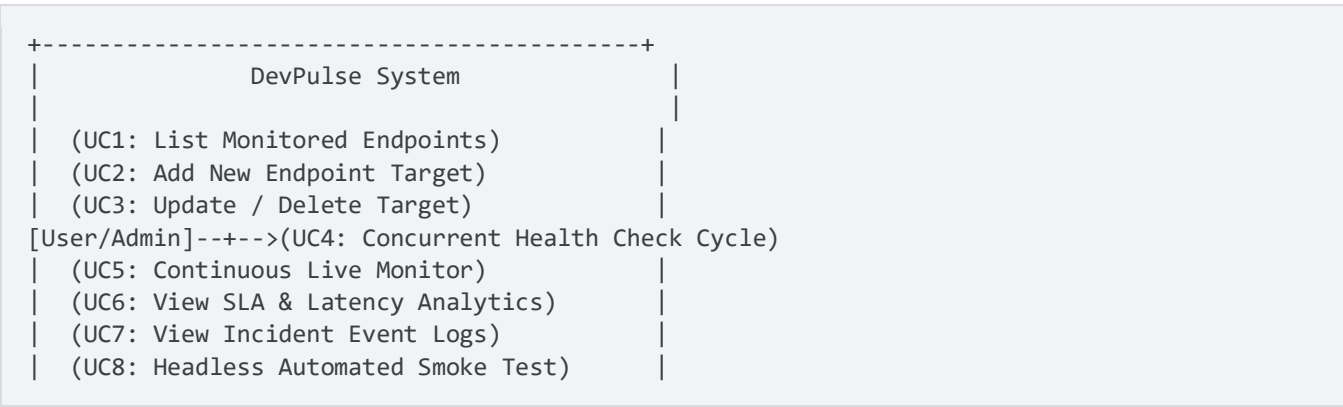
DevPulse follows a strict 3-tier layered architecture, keeping presentation, business logic, and data access cleanly separated:



7. Design Diagrams

7.1 Use Case Diagram

The core interactions available to a system administrator or engineer using DevPulse:



+-----+

7.2 Workflow / Process Flow Diagram

```
[Start App] --> [Initialize Storage & Load data/endpoints.csv]
|
v
[Display Main Menu (Options 0-9)]
|-- [Option 1] --> List Endpoints Table
|-- [Option 2] --> Prompt & Validate Input --> Atomic Save
|-- [Option 3] --> Update Target Fields --> Atomic Save
|-- [Option 4] --> Confirm & Delete --> Atomic Save
|-- [Option 5] --> Concurrently Poll All Endpoints
|                               |-> Compute Latency & Status Code
|                               |-> Record Analytics & Log Failures
|                               |-> Render Aligned ASCII Results Table
|-- [Option 6] --> Start Continuous Monitor Loop
|-- [Option 7] --> Render SLA & Latency Statistics
|-- [Option 8] --> Read Tail of logs/health_events.log
|-- [Option 9] --> Reload Endpoints from Disk
'-- [Option 0 / EOF] --> Shutdown Thread Pool & Exit
```

7.3 Sequence Diagram — Concurrent Health Check Cycle

A simplified trace of what happens between the components when a user triggers option 5, a full concurrent health check:

```
Admin          : Select Option 5
DevPulseApp    -> RegistryService    : getAllEndpoints()
RegistryService -> DevPulseApp        : List<Endpoint>
DevPulseApp    -> PollingEngine       : checkAllConcurrently(endpoints)
PollingEngine  -> Remote APIs         : parallel GET (per endpoint)
Remote APIs    -> PollingEngine       : HTTP response
PollingEngine  -> DevPulseApp         : List<HealthCheckResult>
DevPulseApp    -> AnalyticsService    : recordBatch(results)
AnalyticsService -> FileHandler       : logIncident() [only if DOWN]
AnalyticsService -> DevPulseApp       : done
DevPulseApp    -> Admin              : render results table
```

7.4 Class / Component Diagram

Layer / Package	Class Name	Key Attributes / Methods	Primary Responsibility
com.devpulse.model	Endpoint	id, name, url, expectedStatusCode, timeoutSeconds, toCsv(), fromCsv()	Domain entity representing a monitored target, with RFC-4180 serialisation.
com.devpulse.model	ServiceStatus	UP, DOWN, TIMEOUT, ERROR, getColoredLabel(), isHealthy()	Enumeration of health states with their ANSI colour sequences.

Layer / Package	Class Name	Key Attributes / Methods	Primary Responsibility
com.devpulse.model	HealthCheckResult	endpoint, latencyMs, statusCode, status, message, toLogEntry()	Encapsulates the outcome of a single health probe.
com.devpulse.repository	FileHandler	loadEndpoints(), saveEndpoints(), logIncident(), readRecentLogs()	Data access layer — atomic file swaps and bounded log tailing.
com.devpulse.service	RegistryService	addEndpoint(), updateEndpoint(), deleteEndpoint(), isValidUrl()	Business logic for target registration, bounds checking, and validation.
com.devpulse.service	PollingEngine	checkEndpoint(), checkAllConcurrently(), shutdown()	Multithreaded HTTP polling engine built on CompletableFuture.
com.devpulse.service	AnalyticsService	recordResult(), getGlobalUptimePercentage(), getAllStats()	Computes SLA uptime percentage and latency distribution.
com.devpulse.utils	CLIFormatter	stripAnsi(), padRight(), printHealthCheckResults(), printBanner()	ANSI colour styling and mathematically-aligned ASCII table layouts.
com.devpulse.main	DevPulseApp	main(), run(), readLineSafe(), handleRunHealthCheck()	Interactive CLI entry point that wires the other layers together.

7.5 Storage & Schema Design

DevPulse keeps its state in two lightweight, portable, human-readable files rather than a database — fitting for a zero-dependency CLI tool.

Store 1 — Target Registry (data/endpoints.csv)

Field	Data Type	Constraints	Example
id	String	Primary key, format EP-\d+	EP-101
name	String	Non-empty; quoted if it contains a comma	“Cloudflare Trace”
url	String	Valid HTTP/HTTPS URI	https://1.1.1.1/cdn-cgi/trace
expectedStatusCode	Integer	Range: 100 to 599	200
timeoutSeconds	Integer	Range: 1 to 60	5

Store 2 — Incident Event Audit Log (logs/health_events.log)

Failure events are appended with an ISO-formatted timestamp so they can be searched or replayed later:

```
[2026-09-18 09:52:13] [STATUS: DOWN] [ID: EP-105] [NAME: HTTPBin Simulated Failure]
[URL: https://httpbin.org/status/500] [CODE: 500] [LATENCY: 1290ms]
- Unexpected status: expected 200, got 500
```

8. Design Decisions & Rationale

#	Architectural Decision	Alternatives Considered	Engineering Rationale
1	Zero external dependencies (standard Java 11+)	Apache HttpClient, Jackson, or Retrofit	Compiles on any machine with a JDK, with no build-tool setup and no third-party security surface.
2	Asynchronous parallel polling (CompletableFuture)	Sequential, single-threaded polling	Sequential polling scales as $O(\sum t_i)$; parallel polling scales as $O(\max t_i)$ — 5 pings finish in ~1.3s instead of 5+ seconds.
3	Atomic temp-file swapping	Direct file truncation (TRUNCATE_EXISTING)	Direct writes risk an empty or corrupt config file if the process dies mid-write. Writing to .tmp and swapping via ATOMIC_MOVE keeps the data safe.
4	Memory-bounded circular buffer (ArrayDeque)	Files.readAllLines	readAllLines loads the entire log into heap memory, risking an OutOfMemoryError on large files. A bounded queue keeps memory use constant.
5	Visible-width padding (stripAnsi)	Standard printf string formatting	ANSI escape codes have a non-zero character length but zero visible width, which staggers ASCII borders. Calculating the true visible length keeps tables aligned.

9. Implementation Details

A few of the more interesting algorithmic pieces from the codebase:

9.1 Parallel Polling Engine (PollingEngine.java)

Uses `CompletableFuture.supplyAsync` bound to a custom thread pool sized to the number of available CPU cores (`Math.min(32, cores * 2)`), to dispatch asynchronous HTTP GET requests. Response bodies are discarded with `HttpResponse.BodyHandlers.discarding()` to avoid unnecessary memory allocation — all DevPulse actually needs is the status code and the timing.

9.2 RFC-4180 Compliant CSV Parser (Endpoint.java)

Instead of naively splitting on commas, the parser uses a stateful tokenizer that correctly respects double-quote enclosures (`"..."`) and escaped quotes (`"'"`), so an endpoint name can safely include a comma or a quotation mark without breaking the file.

9.3 Mathematical Visible-Width Alignment (CLIFormatter.java)

Implements `stripAnsi()` using a regular expression (`\u001B\[ [ ; \d ] * m`) to work out the true display width of a string, then pads it to the correct width with `padRight()` so that ANSI-coloured labels never throw off the ASCII table borders.

10. Screenshots & Results

This section walks through DevPulse actually running, captured directly from a live terminal session rather than reconstructed after the fact.

10.1 Application Startup & Main Menu

On launch, DevPulse compiles and prints a startup banner before dropping into its interactive main menu, which lists all nine available operations — from managing endpoints to running a concurrent health check or reviewing the incident log.

```
=====
  Compiling DevPulse: Server & API Health Monitoring CLI...
=====
[SUCCESS] Compilation successful! Launching DevPulse CLI...

=====
                DevPulse Health Monitoring CLI
=====

+----- MAIN MENU -----+
 1. List Registered Endpoints
 2. Add New Endpoint
 3. Update Existing Endpoint
 4. Delete Endpoint
 5. Run Health Check (Concurrent Ping All)
 6. Start Continuous Real-Time Monitor
 7. View Analytics & SLA Summary Report
 8. View Incident Logs (health_events.log)
 9. Reload Configuration from Disk
 0. Exit Application
+-----+

Select an option [0-9]: 1
```

Figure 10.1 — Startup banner and the main menu (Option 1 selected).

10.2 Listing Registered Endpoints

Selecting option 1 prints every currently registered endpoint in a neatly aligned ASCII table — its ID, service name, target URL, expected status code, and timeout — pulled straight from `data/endpoints.csv`.

```
Select an option [0-9]: 1

--- [ REGISTERED ENDPOINTS ] -----
+-----+-----+-----+-----+-----+
| ID      | Service Name      | URL                                     | Expect | Timeout |
+-----+-----+-----+-----+-----+
| EP-101  | Cloudflare Trace  | https://1.1.1.1/cdn-cgi/trace         | 200    | 5s      |
| EP-102  | Google Public DNS | https://dns.google                    | 200    | 5s      |
| EP-103  | JSONPlaceholder API | https://jsonplaceholder.typic...     | 200    | 5s      |
| EP-104  | HTTPBin Success (200) | https://httpbin.org/status/200       | 200    | 5s      |
| EP-105  | HTTPBin Simulated Fai... | https://httpbin.org/status/500       | 200    | 5s      |
+-----+-----+-----+-----+-----+
Total registered endpoints: 5
```

Figure 10.2 — The registry listing for all five seeded endpoints.

10.3 Adding a New Endpoint — Input Validation in Action

Option 2 walks the user through registering a new target. This particular run doubles as a good demonstration of the input validation: typing “default 200” instead of a plain number for the expected status code is correctly rejected, since RegistryService insists on strictly numeric input for the status code and timeout fields.

```
Select an option [0-9]: 2

--- [ ADD NEW ENDPOINT ] -----
Enter descriptive name (e.g. Payments Gateway): Payments Gateway
Enter target URL (e.g. https://api.example.com/health): https://api.example.com/health
Enter expected HTTP status code [default 200]: default 200
Error: Numeric input required for status code and timeout.
```

Figure 10.3 — Add Endpoint flow catching a non-numeric status code before it can be saved.

10.4 Updating an Existing Endpoint

Option 3 reuses the same registry table so the user can pick a target to edit, then updates only the fields that are changed, leaving everything else untouched.

```
Select an option [0-9]: 3

--- [ UPDATE ENDPOINT ] -----
+-----+-----+-----+-----+-----+
| ID      | Service Name      | URL                                     | Expect | Timeout |
+-----+-----+-----+-----+-----+
| EP-101  | Cloudflare Trace  | https://1.1.1.1/cdn-cgi/trace         | 200    | 5s      |
| EP-102  | Google Public DNS | https://dns.google                    | 200    | 5s      |
| EP-103  | JSONPlaceholder API | https://jsonplaceholder.typic...     | 200    | 5s      |
| EP-104  | HTTPBin Success (200) | https://httpbin.org/status/200       | 200    | 5s      |
| EP-105  | HTTPBin Simulated Fai... | https://httpbin.org/status/500       | 200    | 5s      |
+-----+-----+-----+-----+-----+
Total registered endpoints: 5
```

Figure 10.4 — Update Endpoint screen showing the current registry before an edit.

10.5 Concurrent Health Check Execution

This is the heart of the application. Option 5 fires off all five endpoints at once using the asynchronous polling engine. In this run, four targets (including the deliberately unreliable HTTPBin “success” endpoint) came back UP within around

1.3 seconds combined, while the endpoint seeded to simulate a failure (HTTPBin Simulated Failure, expecting a 500) is correctly flagged as DOWN with full diagnostics.

```
Select an option [0-9]: 5

--- [ CONCURRENT HEALTH CHECK CYCLE ] -----
Polling 5 endpoints asynchronously in parallel...

+-----+-----+-----+-----+-----+-----+-----+
| ID      | Service Name      | Status      | Code  | Latency  | Time       | Diagnostics      |
+-----+-----+-----+-----+-----+-----+-----+
| EP-101  | Cloudflare Trace  | [OK] UP     | 200   | 780 ms   | 10:33:19   | Healthy (HTTP 200) |
| EP-102  | Google Public DNS | [OK] UP     | 200   | 780 ms   | 10:33:19   | Healthy (HTTP 200) |
| EP-103  | JSONPlaceholder API | [OK] UP     | 200   | 864 ms   | 10:33:19   | Healthy (HTTP 200) |
| EP-104  | HTTPBin Success (200) | [OK] UP     | 200   | 1336 ms  | 10:33:19   | Healthy (HTTP 200) |
| EP-105  | HTTPBin Simulated F... | [FAIL] DOWN | 500   | 1336 ms  | 10:33:19   | Unexpected status: ex... |
+-----+-----+-----+-----+-----+-----+-----+

Total cycle execution time: 1360 ms (parallel concurrency achieved)
```

Figure 10.5 — A full concurrent health check cycle across all five registered endpoints.

11. Testing Approach

DevPulse ships with its own automated verification suite (com.devpulse.test.DevPulseTest). All eight test cases pass cleanly:

```
=====
Running DevPulse Comprehensive Test Suite
=====
[PASS] testEndpointModelAndCsvRFC4180
[PASS] testUrlValidationAndBoundaries
[PASS] testFilePersistenceAndAtomicWrite
[PASS] testMemoryBoundedLogReading
[PASS] testAnalyticsAndSlaCalculation
[PASS] testAnalyticsEndpointSync
[PASS] testCliFormatterVisualPadding
[PASS] testPollingEngineConcurrency
=====
Tests Completed: 8 Passed, 0 Failed
=====
```

12. Challenges Faced & Engineering Solutions

- 1. **Invisible ANSI colour codes misaligning tables:** Standard printf counts invisible escape codes toward a string’s length, which staggers table borders. Solved by writing stripAnsi() and padRight() to calculate the actual visible width.
- 2. **Atomic file persistence across platforms:** Directly overwriting a file on Windows can collide with existing filesystem locks. Solved with atomic temporary-file writes plus a fallback replacement path.
- 3. **Unbounded thread spawning under heavy load:** An unbounded thread pool risked exhausting OS threads if too many targets were registered. Solved by capping the pool size to the number of available processor cores.
- 4. **Scanner crashing on EOF:** The standard Scanner.nextLine() throws a NoSuchElementException when the input stream ends or the app runs non-interactively. Solved with a defensive readLineSafe() helper.

13. Learnings & Key Takeaways

- **Modern Java concurrency, hands-on:** Real practical experience with `CompletableFuture`, `ExecutorService`, and non-blocking HTTP clients for parallel network I/O.
- **Clean architecture pays off:** Keeping the Model, Repository, Service, and Presentation layers decoupled made the codebase noticeably easier to extend and to test.
- **Defensive programming as a habit:** Designing for failure — validating boundaries, handling timeouts, and adding atomic persistence — makes a real difference to how resilient the app feels under stress.
- **Terminal UX still matters:** A well-formatted CLI can deliver a clear, intuitive experience without needing the overhead of a full graphical framework.

14. Future Enhancements

- **Multi-channel webhook notifications:** Push instant alerts to Slack, Discord, or Microsoft Teams the moment a service goes DOWN.
- **Prometheus & OpenTelemetry metrics export:** Expose a lightweight metrics endpoint that enterprise observability platforms can scrape directly.
- **HTTP POST payload & header customisation:** Support custom JSON payloads and authorization headers for authenticated microservice endpoints.
- **SSL expiry auditing:** Proactively warn when a monitored endpoint's SSL/TLS certificate is about to expire.

15. References

1. Shafranovich, Y. (2005). *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180, Network Working Group.
2. Oracle Corporation. (2024). *Java Platform, Standard Edition & Java Development Kit Version 11/21/25 API Specification — `java.net.http.HttpClient`*.
3. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
4. Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.
5. Fielding, R. T., & Taylor, R. N. (2002). *Principled Design of the Modern Web Architecture*. ACM Transactions on Internet Technology (TOIT), 2(2), 115-150.